

Customer Group Predictor — Case Study

Problem Statement

A marketplace runs **paired marketing campaigns**: every round, two customer groups are compared and only one is most profitable to target. Targeting the wrong group wastes spend.

The naive heuristic — “*always pick Group 1*” — is right **46.5%** of the time. The goal is a model that beats this baseline **materially** and is deployed end-to-end: from raw CSV, through a warehouse and feature pipeline, to a versioned model served behind a low-latency API, with scheduled retraining and the monitoring needed to catch regressions before they hit revenue.

The deliverable is therefore **two things at once**:

1. A model that is measurably better than the baseline on held-out data.
2. A system that keeps it that way — reproducible training, gated promotion, auto-redeploy, and observability.

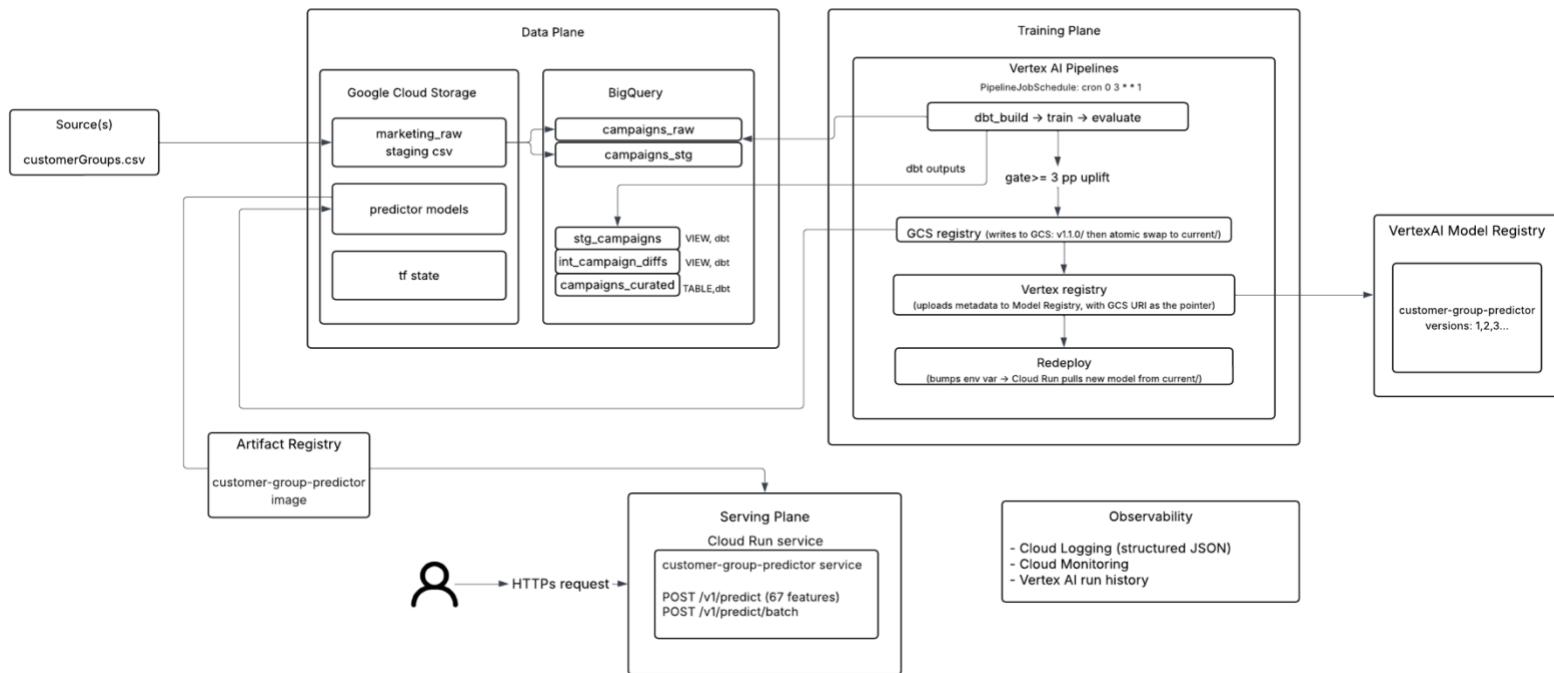
What the data says

Outcome	Campaigns	Share
Group 1 most profitable	3,076	46.47%
Group 2 most profitable	1,877	28.35%
Neither profitable	1,667	25.18%
Total	6,620	100%

- **n = 6,620** historical campaigns
- **67 pre-campaign features** per row (g1_1...g1_20, g2_1...g2_20, c_1...c_27)
- **3 post-campaign columns** (g1_21, g2_21, c_28) — outcome leakage, excluded from training and rejected by the API schema
- Strong **positional bias**: Group 1 wins $\sim 1.64\times$ as often as Group 2, which the model must not learn as a shortcut
- Three-class target — Neither profitable is a real outcome, not noise, so the model treats it as a first-class class rather than collapsing it into a binary

The class imbalance (46/28/25) and the positional bias together drive two of the most important design choices: **swap augmentation** during training, and **lift-over-baseline** (not raw accuracy) as the promotion gate.

High-level system architecture



Architecture

(Reference: /docs/customer-group-predictor.png)

Sequence

Online prediction path : (detailed reference: /docs/sequence_diag_online_prediction.png)

Caller (marketer/UI)

POST /v1/predict { g1_*, g2_*, c_* }

Cloud Run (FastAPI)

1. Pydantic schema validation
 - rejects extra fields (extra="forbid")
 - rejects post-campaign cols (g1_21, g2_21, c_28) → 422
2. FeatureBuilder.transform(df)
 - re-checks leakage guard
 - computes diff features $d_i = g1_i - g2_i$ (20 features)
 - computes ratio features $r_i = g1_i / (g2_i + \epsilon)$ (20 features)
 - returns deterministic 107-col matrix
3. ModelLoader (singleton)
 - holds in-memory XGBoost booster

- loaded from GCS at container start

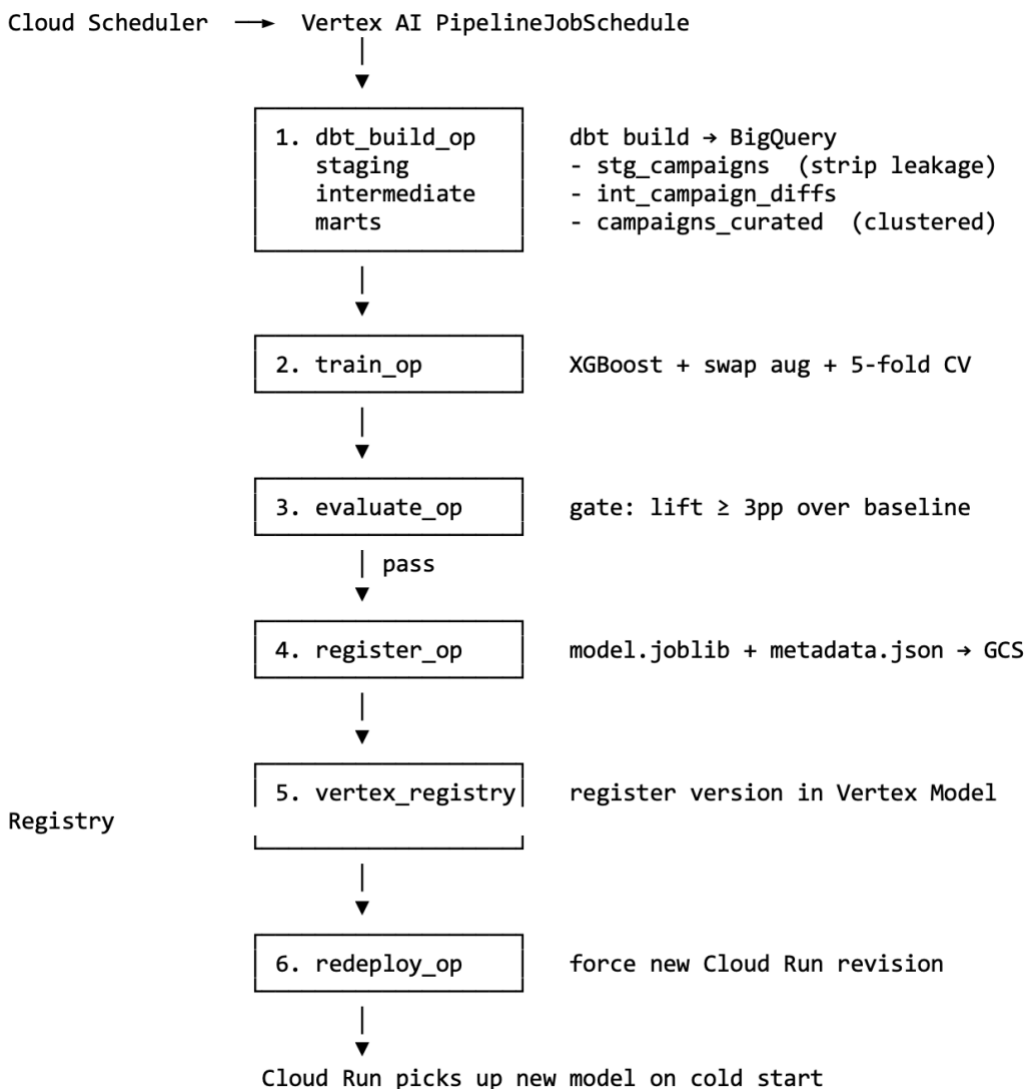
4. `booster.predict_proba(X) → [p0, p1, p2]`

5. Response: { winner: "GROUP_1", probabilities: {...},
model_version: "1.1.0", request_id: ... }

▼

Caller ← 200 OK (p50 ~25 ms, p95 ~80 ms)

Offline training path (weekly, Mon 03:00 UTC) :
(detailed reference: /docs/sequence__diag_scheduled_training.png)



If the evaluate gate fails, the pipeline halts at step 3 — the previous model stays in production and an alert fires. **No silent regressions.**

Choice of Tech Stack

Choice	Why
All-GCP stack (BigQuery, Cloud Run, Vertex AI, GCS, Artifact Registry)	Matches Payback's existing tech stack—no new vendor onboarding, unified IAM, and a single billing surface. Managed services keep the operational footprint small for a one-engineer build. Used a new Google account for the initial \$300 free credit.
dbt for transformations	SQL as code with version control, schema tests, and column-level lineage. The same transformation logic runs in development and production against BigQuery.
XGBoost (vs. Logistic Regression / Random Forest)	Best test-set lift (+7.48 pp over the 46.45% baseline), tolerates correlated features (the d_{*}/r_{*} engineered features are highly collinear with $g1_{*}/g2_{*}$), trains in seconds on 6,620 rows, and produces a ~5 MB artifact that loads quickly on Cloud Run cold starts.
Terraform with GCS remote state	Infrastructure as code—every resource (BigQuery datasets, Cloud Run service, IAM bindings, Artifact Registry, and GCS buckets) is reviewable in a pull request. A versioned GCS bucket enables team collaboration and safe terraform plan execution from any machine.
FastAPI	Async-first Python web framework with Pydantic v2 built in. The same schema layer that validates <code>/v1/predict</code> payloads also generates OpenAPI docs at <code>/docs</code> . <code>extra="forbid"</code> on the request model acts as the runtime leakage guard. It offers typed responses, low boilerplate, and a Cloud Run-friendly deployment model.
Vertex AI Pipelines (vs. Cloud Scheduler + Cloud Run Jobs)	Native step-level lineage, artifact tracking, and integration with Vertex Model Registry. Each pipeline run is a first-class object that can be inspected, rerun from a specific step, and rolled back from. Built-in <code>PipelineJobSchedule</code> removes the need for a separate scheduler.
Cloud Run (vs. GKE / Cloud Functions)	Scale-to-zero serverless containers—no idle cost when traffic is low and automatic scaling during bursts. The 5 MB model and FastAPI service fit comfortably in a single revision; cold start is ~2 seconds.
Docker	Builds and runs the image consistently, especially in combination with the Makefile.
Makefile	A single Makefile documents every workflow— <code>make install</code> , <code>make test</code> , <code>make run</code> , and <code>make deploy-all</code> . New contributors do not need to learn the internals first; <code>make help</code> serves as the table of contents. Targets are dependency-tracked so <code>make run</code> rebuilds the model only when sources change.

Components

The system has five layers. Together, they provide a predictable path from raw CSV data to a live prediction, and from a code change to a fully retrained, redeployed model on a schedule.

1. Data layer — GCS+BigQuery + dbt

This is where the raw data lands and gets shaped into something a model can train on. **dbt** sits on top of BigQuery and turns the transformations into versioned, tested SQL — the same way application code is reviewed and tested in a PR.

- **marketing_raw** – assuming the source data is dumped into a location on GCS
- **raw_campaigns** - the original customerGroups.csv loaded as-is. Append-only.
- **stg_campaigns** (*staging*) - de-duplicated; **the post-campaign columns are dropped here**. This is our first guard against the model accidentally learning from outcome data it would never have at prediction time.
- **int_campaign_diffs** (*intermediate*) - adds the engineered features (40 columns derived from the raw inputs). The SQL mirrors the Python feature code exactly, so training and serving see the same numbers.
- **campaigns_curated** (*final mart*) - the table the trainer actually reads. Clustered on the target column for fast scans.
- **17 dbt tests** - uniqueness, not-null, accepted value ranges, plus a custom test that fails the build if any post-campaign column reappears. Bad data never reaches training silently.

2. Training layer — Vertex AI Pipelines

The training pipeline is six steps wired together in Vertex AI Pipelines. Each step is a separate container with a single job, so any failure is isolated and re-runnable.

1. **Refresh data (dbt build)** - rebuilds the curated mart and runs all 17 dbt tests. If the data is malformed, the pipeline stops here.
2. **Train** - pulls the curated table, applies **swap augmentation** (mirror every row with Group 1/2 swapped) to remove a positional bias in the raw data, runs **5-fold stratified cross-validation**, and fits the final XGBoost on the full training set. A logistic regression is also fit as a sanity-check baseline.
3. **Evaluate** - measures lift over the 46.45% always-pick-Group-1 baseline on a held-out split. **Promotion gate: lift must be ≥ 3 pp**. If the new model isn't materially better, the pipeline fails and the old model stays serving.
4. **Register artifact** - saves the model file plus a metadata JSON (version, training commit SHA, CV scores, feature list, SHA-256 of the model file) to a versioned GCS path. Every artifact is fully traceable.
5. **Vertex Model Registry** — registers the new model version for lineage and one-click rollback.
6. **Redeploy** — triggers a new Cloud Run revision so the next request picks up the new model. Traffic shift is atomic — no half-rolled-out state.

Schedule: runs every Monday at 03:00 UTC. **End-to-end runtime:** ~8 minutes.

Therefore, retraining is automated, gated, and fully audited. A bad model can't reach production without a human override.

3. Serving layer — FastAPI on Cloud Run

The prediction service is a stateless FastAPI app running on Cloud Run. It loads the model once at startup and stays warm; cold starts are ~ 2 seconds.

- **Strict input validation** — the request schema rejects unknown fields with a clear 422 error. Any caller that accidentally includes a post-campaign column is blocked at the door, not silently scored.
- **Same feature code as training** — the serving path imports the exact same Python module the trainer used.
- **Endpoints:**
 - GET /healthz — liveness (is the process up?)
 - GET /readyz — readiness (is the model loaded?)
 - GET /v1/model/info — current model version, training commit SHA, feature list
 - POST /v1/predict — the main prediction call
 - POST /v1/predict/batch — batch prediction call
- **Autoscaling** — Cloud Run scales from 0 to 10 instances based on traffic. Zero idle cost when nothing is calling.

4. Observability

- **Structured logs** — every request is logged with a request ID, the model version that served it, the predicted class, and latency. Searchable in Cloud Logging.

5. Security and reproducibility

- **Infrastructure as code** — every GCP resource is defined in Terraform with remote state. The entire stack can be torn down and rebuilt from a clean project in one command.
- **Full lineage** — every production prediction can be traced back to: a specific Cloud Run revision → a specific Docker image (tagged with semantic version + git SHA) → a specific model file (SHA-256 in the metadata) → a specific training run → a specific dbt build → a specific row count in the curated mart.

If something goes wrong, we can answer which model produced this prediction, on what data, from what code, and roll back to the previous good state in seconds.

Approach to training

Why XGBoost

The bar to clear was 46.45% — the accuracy of "always pick Group 1". We tested three model families against it on the same data, using **5-fold cross-validation** (the standard way to estimate how a model will perform on data it hasn't seen):

Model	Accuracy	Lift over Baseline	One-Line Take
“Always Group 1”	46.45%	0.00 pp	The bar to clear
Logistic regression	50.1%	+3.6 pp	Simple and fast, but assumes linear patterns
Random Forest	52.4%	+5.9 pp	Strong, but slower and about 10× larger to ship
XGBoost	53.3%	+6.87 pp	Winner — best accuracy and fast

Why XGBoost: it captures interactions between features (combinations of inputs that matter together, not just individually), handles the slight class imbalance well, and produces a small model file that loads quickly when the service scales up. Same family of models powers most production tabular-data ML at companies like Uber, Airbnb, and across fintech.

Feature Engineering

The raw data gives us 67 "before the campaign" numbers per row (20 about Group 1, 20 about Group 2, 27 comparison features). We feed the model those plus **40 derived features** that make the comparison between the two groups explicit:

- **20 difference features** $d_i = g1_i - g2_i$: "How much higher is Group 1 than Group 2 on this attribute?"
- **20 ratio features** $r_i = g1_i / g2_i$ "How many times bigger is Group 1's number than Group 2's?"

The model could in principle learn these on its own, but **handing them to it directly** lets it find the right answer faster with the data we have (6,620 rows is not huge).

One source of truth. The Python code that builds these features at training time is the same code the live API uses to build them at prediction time.

Swap Augmentation

Group 1 wins **1.64×** as often as Group 2 in the raw data. But this is not because Group 1 is genuinely better. It's because of how the rows were ordered when the dataset was assembled. A naive model would learn the unhelpful shortcut *"when unsure, pick Group 1"*, which is exactly the heuristic we're trying to beat.

The fix: for every row in training, we add a mirror row with Group 1 and Group 2 swapped (and the answer flipped to match). This:

- doubles the training data to ~13,240 rows,
- balances the wins between the two groups, and
- forces the model to learn the **real comparison** rather than memorizing row order.

Measured impact: **+1.4 pp extra lift** compared to training without this trick.

Validation Strategy

- **5-fold cross-validation** - we split the data into 5 chunks, train on 4, test on the 5th, rotate, and average. This gives a robust accuracy estimate, not a number that got lucky on one split.
- **Stratified splits** - each chunk preserves the same Group 1 / Group 2 / Neither mix as the full dataset, so we never evaluate on a fold that happens to have unusually many easy or hard examples.
- **The mirror-row trick is only applied to training data, never to test data** - otherwise it could give the model a peek at the answer and inflate the score.
- **The headline number is *lift over the baseline*, not raw accuracy.** With an imbalanced 3-class problem, raw accuracy can look good even when the model has learned nothing. Lift answers the only question a business actually cares about: *how much better than the obvious strategy of Group 1 being the default pick?*

Leakage Defense:

The biggest risk in this dataset is that a post-campaign column (g1_21, g2_21, or c_28) sneaks into features and produces an “amazing” model that would fail in production, because at real prediction time, you don't have the campaign result yet. We guard against this at **four independent layers**:

The dataset includes three "after-the-campaign" columns (the campaign results themselves). If those ever leak into training, the model would look amazing on tests and fail completely in production — because at real prediction time, you don't have the campaign result yet. We guard against this at **four independent layers**:

1. **The SQL transformation** (dbt staging SQL) explicitly drops the post-campaign columns.
2. **A dedicated data test** (dbt singular test) fails the build if they reappear.
3. **The Python feature code** refuses to run if it sees them. **FeatureBuilder.validate** raises `LeakageError` if any post-campaign column is present in its input `DataFrame`.
4. **The Pydantic API request schema** rejects them with a clear 422 error on `/v1/predict`.

Promotion Gate

When the weekly retraining runs, the new model is **only promoted to production if it beats the baseline by at least 3 pp and matches or beats the current model**. If it doesn't, the pipeline halts, the old model keeps serving, and the team is alerted to investigate.

This is what makes the schedule safe to leave on autopilot: a degraded model can't sneak into production just because it's the freshest one trained.

Limitations and next priorities

With more time and resources, the priorities below would close the gaps and operate safely at scale on production:

- **CI/CD pipeline:** Today, build/test/deploy is driven by make targets run locally. The next step is a GitHub Actions pipeline that: lints, runs the 29 unit/integration tests + 17 dbt tests on every PR, builds the linux/amd64 image, pushes to Artifact Registry on merge to main, and triggers terraform apply against a dev project before promotion to prod. Image tags would be <semver>-<git-sha> so every Cloud Run revision is traceable to a commit.
- **Observability via a dedicated tool (Datadog or equivalent):** Cloud Logging + Cloud Monitoring covers the basics but is GCP-siloed. A Datadog integration would give us unified APM traces across Cloud Run, BigQuery, and Vertex, log-based alerts with richer routing (Teams Channel or Slack), and a single dashboard for SLOs (p95 latency, error rate, prediction-class drift). Especially valuable once this service sits next to others in a wider product surface.
- **Single-region serving:** Cloud Run is regional (europe-west3). For multi-region failover, the next iteration would put a global HTTPS load balancer in front of regional Cloud Run services in (at least) europe-west3 and us-central1, with serverless NEG's and Cloud Armor for WAF. This also unlocks lower latency for non-EU callers.
- **Authentication:** The /v1/predict endpoint is intentionally left open for ease of evaluation. In production it would sit behind one of: (a) Cloud Run IAM invoker with service-account-to-service-account auth for internal callers; (b) API Gateway + API keys for first-party clients; or (c) Identity-Aware Proxy (IAP) for human users. Combined with Cloud Armor rate-limiting to blunt abuse.
- **Drift-triggered retraining:** Retraining is currently time-based (Mondays 03:00 UTC). A data-drift detector on the input feature distribution would let us retrain when the world changes, not on a fixed cadence. Implementation: a lightweight Cloud Run Job runs nightly against BigQuery, computes drift scores, and either fires an alert or triggers the Vertex pipeline ad-hoc when thresholds are breached.

Reference:

- Postman collection: [custgroupprediction/docs/postman collection/postman_collection.json](#)
- The service has been deployed and exposed on the following URL:
<https://customer-group-predictor-95597606705.europe-west3.run.app>